

循序渐进，学习开发 xv6-riscv

第 16 章 系统调用

汪辰



目录

01

系统调用初探

02

xv6 中的系统调用函数

03

xv6 系统调用的设计与实现



01

系统调用初探

系统调用初探

- 系统调用是什么
- 系统调用的执行流程

系统调用初探

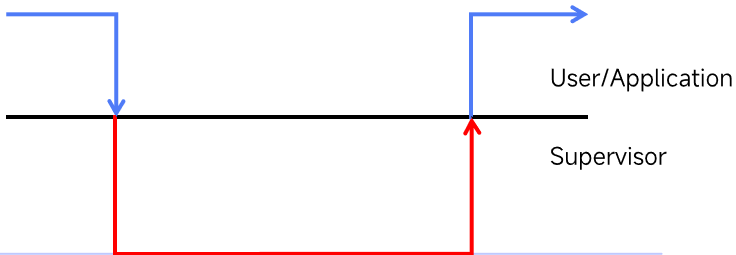
- **系统调用是什么**
- 系统调用的执行流程

系统调用是什么

Table 1. RISC-V privilege levels.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	<i>Reserved</i>	
3	11	Machine	M

The RISC-V Instruction Set Manual Volume II , Chapter 1.2. Privilege Levels

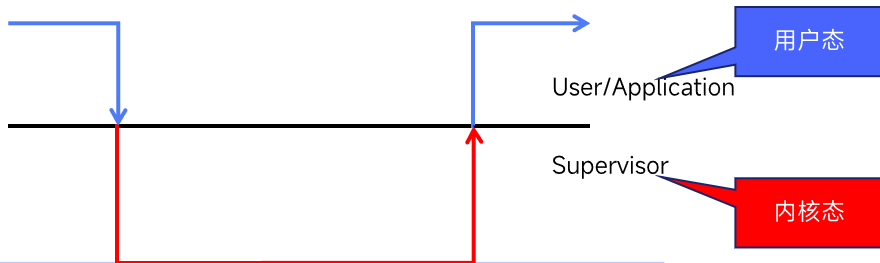


系统调用是什么

Table 1. RISC-V privilege levels.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	
3	11	Machine	M

The RISC-V Instruction Set Manual Volume II , Chapter 1.2. Privilege Levels

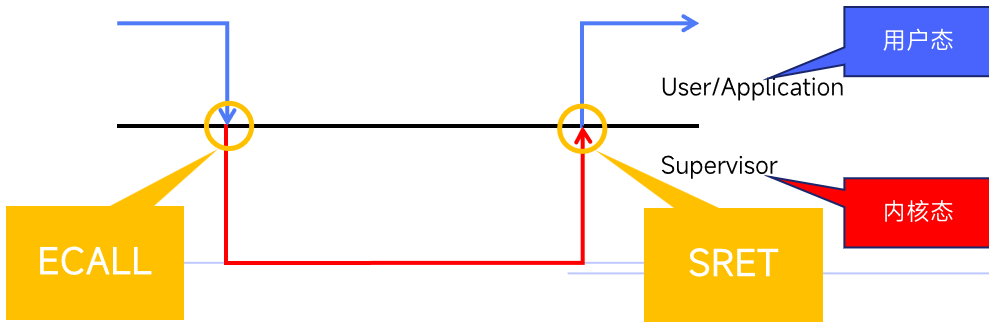


系统调用是什么

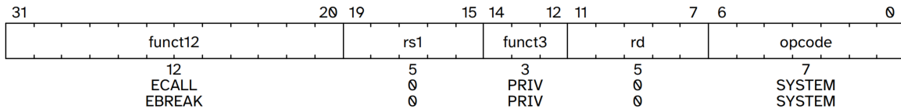
Table 1. RISC-V privilege levels.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	
3	11	Machine	M

The RISC-V Instruction Set Manual Volume II , Chapter 1.2. Privilege Levels



系统调用是什么



The RISC-V Instruction Set Manual Volume II , 3.3.1. Environment Call and Breakpoint

- ECALL 命令用于主动触发异常
- 根据调用 ECALL 的权限级别产生不同的 exception code
- 异常产生时 epc 寄存器的值存放的是 ECALL 指令本身的地址。

Interrupt	Exception Code	Description
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10-11	Reserved
0	12	Instruction page fault
0	13	Load page fault
0	14	Reserved
0	15	Store/AMO page fault
0	16-17	Reserved
0	18	Software check
0	19	Hardware error
0	20-23	Reserved
0	24-31	Designated for custom use
0	32-47	Reserved
0	48-63	Designated for custom use
0	≥64	Reserved

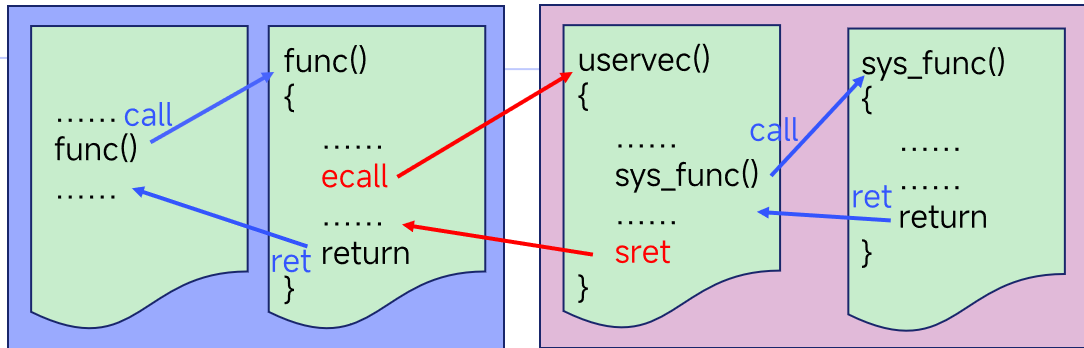
系统调用初探

- 系统调用是什么
- **系统调用的执行流程**

系统调用的执行流程

用户 (User) 态

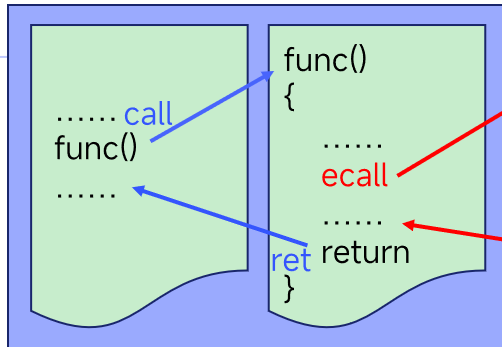
内核 (Supervisor) 态



系统调用的执行流程

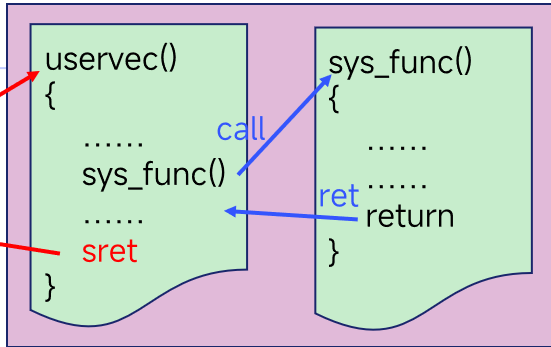
用户 (User) 态

内核 (Supervisor) 态



user/usys.S

```
.global write
write:
    li a7, SYS_write
    ecall
    ret
```



kernel/sysfile.c

```
uint64 sys_write(void)
{
    int n;
    uint64 p;
    //.....
}
```



02

xv6 中的系统调用函数



xv6 系统调用一览

```
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int*);
int kill(int);
int exec(const char*, char**);
int getpid(void);
char* sys_sbrk(int, int);
int pause(int);
int uptime(void);
```

```
int open(const char*, int);
int read(int, void*, int);
int write(int, const void*, int);
int close(int);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int mknod(const char*, short, short);
int pipe(int*);
```

xv6 系统调用一览

```
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int*);
int kill(int);
int exec(const char*, char**);
int getpid(void);
char* sys_sbrk(int, int);
int pause(int);
int uptime(void);
```

进程相关

```
int open(const char*, int);
int read(int, void*, int);
int write(int, const void*, int);
int close(int);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int mknod(const char*, short, short);
int pipe(int*);
```

xv6 系统调用一览

```
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int*);
int kill(int);
int exec(const char*, char**);
int getpid(void);
char* sys_sbrk(int, int);
int pause(int);
int uptime(void);
```

进程相关

```
int open(const char*, int);
int read(int, void*, int);
int write(int, const void*, int);
int close(int);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int mknod(const char*, short, short);
int pipe(int*);
```

文件相关

A decorative graphic on the left side of the slide. It features two concentric circles with a light blue glow. A thin grey arc starts from the top of the outer circle and curves around to the bottom. In the top right corner, there is a solid blue rectangle.

03

xv6 系统调用的设计与实现

xv6 系统调用的设计与实现

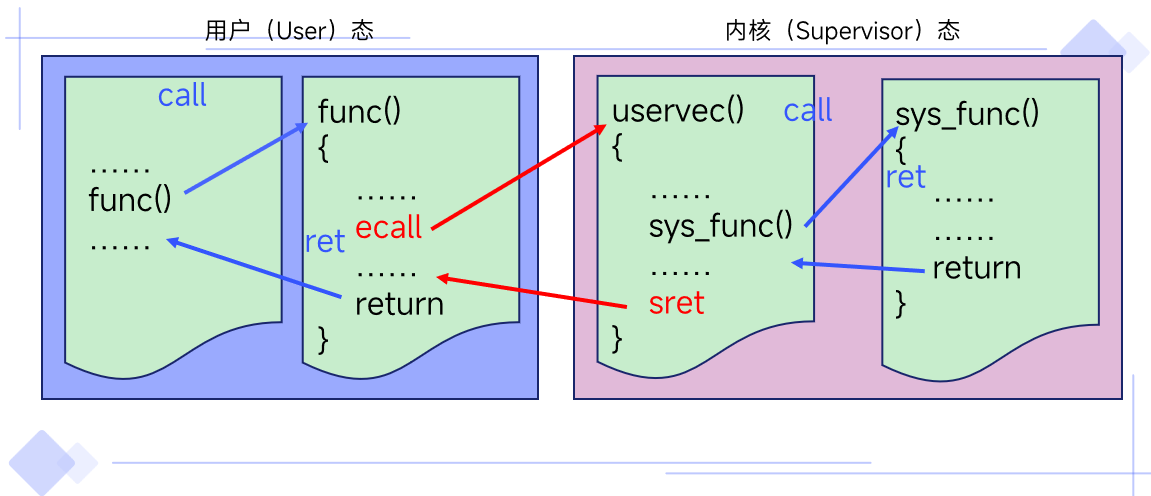
- xv6 系统调用接口框架的实现
- 第一个系统调用 write
- 完善系统调用框架
- 第二个系统调用 getpid

xv6 系统调用的设计与实现

- **xv6 系统调用接口框架的实现**

- 第一个系统调用 write
- 完善系统调用框架
- 第二个系统调用 getpid

xv6 系统调用接口框架的实现



xv6 系统调用接口框架的实现

- 系统调用作为操作系统的对外接口，由操作系统的实现负责定义。
- xv6-riscv 的系统调用的传参约定参考了 Linux
 - ✓ 系统调用号放在 a7 中，用于区分不同的系统调用功能。
 - ✓ 系统调用参数最多 6 个，依次使用 a0 ~ a5
 - ✓ 返回值使用 a0

xv6 系统调用接口框架的实现

kernel/syscall.h

```
#define SYS_fork      1
#define SYS_exit      2
#define SYS_wait      3
#define SYS_pipe      4
#define SYS_read       5
#define SYS_kill       6
#define SYS_exec       7
#define SYS_fstat      8
#define SYS_chdir      9
#define SYS_dup       10
#define SYS_getpid     11
#define SYS_sbrk       12
#define SYS_pause      13
#define SYS_uptime     14
#define SYS_open       15
#define SYS_write      16
#define SYS_mknod      17
#define SYS_unlink     18
#define SYS_link       19
#define SYS_mkdir      20
#define SYS_close      21
```

xv6 系统调用接口框架的实现

kernel/syscall.h

```
#define SYS_fork      1
#define SYS_exit      2
#define SYS_wait      3
#define SYS_pipe      4
#define SYS_read      5
#define SYS_kill      6
#define SYS_exec      7
#define SYS_fstat     8
#define SYS_chdir     9
#define SYS_dup      10
#define SYS_getpid    11
#define SYS_sbrk     12
#define SYS_pause     13
#define SYS_uptime    14
#define SYS_open      15
#define SYS_write     16
#define SYS_mknod     17
#define SYS_unlink    18
#define SYS_link      19
#define SYS_mkdir     20
#define SYS_close     21
```

diff --git a/kernel/trap.c b/kernel/trap.c

index 079470c..1465867 100644

--- a/kernel/trap.c

+++ b/kernel/trap.c

@@ -50,7 +50,16 @@ usertrap(void)

```
    if(r_scause() == 8){
        // system call
        -    panic("usertrap: syscall not implemented yet");
        +
        +    // sepc points to the ecall instruction,
        +    // but we want to return to the next instruction.
        +    p->trapframe->epc += 4;
        +
        +    // an interrupt will change sepc, scause, and sstatus,
        +    // so enable only now that we're done with those registers.
        +    intr_on();
        +
        +    syscall();
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
```

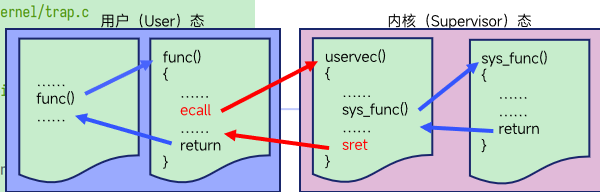
xv6 系统调用接口框架的实现

kernel/syscall.h

```
#define SYS_fork      1
#define SYS_exit      2
#define SYS_wait      3
#define SYS_pipe      4
#define SYS_read       5
#define SYS_kill       6
#define SYS_exec       7
#define SYS_fstat      8
#define SYS_chdir      9
#define SYS_dup       10
#define SYS_getpid     11
#define SYS_sbrk       12
#define SYS_pause      13
#define SYS_uptime     14
#define SYS_open       15
#define SYS_write      16
#define SYS_mknod      17
#define SYS_unlink     18
#define SYS_link       19
#define SYS_mkdir      20
#define SYS_close      21
```

```
diff --git a/kernel/trap.c b/kernel/trap.c
index 079470c..1465867 100644
--- a/kernel/trap.c
+++ b/kernel/trap.c
@@ -50,7 +50,16 @@ usertrap(void)
{
    if(r_scause() == 8){
        // system call
        panic("usertrap: syscall r")

-
+
+    // sepc points to the ecall instruction,
+    // but we want to return to the next instruction.
+    p->trapframe->epc += 4;
+
+    // an interrupt will change sepc, scause, and sstatus,
+    // so enable only now that we're done with those registers.
+    intr_on();
+
+    syscall();
    } else if((which_dev = devintr()) != 0){
        // ok
    } else {
```



xv6 系统调用接口框架的实现

kernel/syscall.h

```
#define SYS_fork      1
#define SYS_exit      2
#define SYS_wait      3
#define SYS_pipe      4
#define SYS_read       5
#define SYS_kill       6
#define SYS_exec       7
#define SYS_fstat      8
#define SYS_chdir      9
#define SYS_dup       10
#define SYS_getpid     11
#define SYS_sbrk       12
#define SYS_pause      13
#define SYS_uptime     14
#define SYS_open        15
#define SYS_write       16
#define SYS_mknod       17
#define SYS_unlink      18
#define SYS_link        19
#define SYS_mkdir       20
#define SYS_close       21
```

```
diff --git a/kernel/trap.c b/kernel/trap.c
index 079470c..1465867 100644
```

```
--- a/kernel/trap.c
+++ b/kernel/trap.c
@@ -50,7 +50,16 @@ usertrap(void)

    if(r_scause() == 8){
        // system call
        panic("usertrap: syscall not implemented");
    }
    // sepc points to the ecall instruction,
    // but we want to return to the next instruction
    p->trapframe->epc += 4;

    // an interrupt will change sepc, so cause
    // so enable only now that we're done with the syscall
    intr_on();

    syscall();
} else if((which_dev = devintr()) != 0){
    // ok
} else {
```

kernel/syscall.c

```
// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
    // For example, to be added soon:
    // [SYS_write] sys_write,
};

void
syscall(void)
{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->a7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        // Use num to lookup the system call function for num, call it,
        // and store its return value in p->trapframe->a0
        p->trapframe->a0 = syscalls[num]();
    } else {
        printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}
```

xv6 系统调用接口框架的实现

```
// Fetch the nth 32-bit system call argument.
void argint(int n, int *ip)
{
    *ip = argraw(n);
}

// Retrieve an argument as a pointer.
// Doesn't check for legality, since
// copyin/copyout will do that.
void argaddr(int n, uint64 *ip)
{
    *ip = argraw(n);
}
```

```
static uint64 argraw(int n)
{
    struct proc *p = myproc();
    switch (n) {
        case 0:
            return p->trapframe->a0;
        case 1:
            return p->trapframe->a1;
        case 2:
            return p->trapframe->a2;
        case 3:
            return p->trapframe->a3;
        case 4:
            return p->trapframe->a4;
        case 5:
            return p->trapframe->a5;
    }
    panic("argraw");
    return -1;
}
```

xv6 系统调用的设计与实现

- xv6 系统调用接口框架的实现
- **第一个系统调用 write**
- 完善系统调用框架
- 第二个系统调用 getpid

第一个系统调用 write

```
write(2)                                System Calls Manual                                write(2)
NAME
    write - write to a file descriptor
LIBRARY
    Standard C library (libc, -lc)
SYNOPSIS
    #include <unistd.h>

    ssize_t write(int fd, const void buf[.count], size_t count);
```

man 2 write

第一个系统调用 write

write(2) System Calls Manual write(2)

NAME

write - write to a file descriptor

LIBRARY

Standard C library (libc)

SYNOPSIS

#include <unistd.h>

ssize_t write(int fd, const void buf[.count], size_t count);

user/user.h

int write(int fd, const void *buf, int count);

第一个系统调用 write

write(2) System Calls Manual write(2)

NAME

write - write to a file descriptor

LIBRARY

Standard C library (libc)

SYNOPSIS

```
#include <unistd.h>
```

```
ssize_t write(int fd, const void buf[.count], size_t count);
```

user/user.h

```
int write(int fd, const void *buf, int count);
```

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index ac26e3e..499c8f9 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -45,11 +45,13 @@ argaddr(int n, uint64 *ip)
     *ip = argraw(n);
 }

+// Prototypes for the functions that handle system calls.
+extern uint64 sys_write(void);
+
+// An array mapping syscall numbers from syscall.h
+// to the function that handles the system call.
+static uint64 (*syscalls[])(void) = {
-// For example, to be added soon:
-// [SYS_write] sys_write,
+[SYS_write] sys_write,
};
```

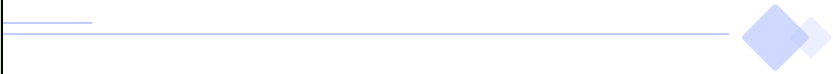
```
diff --git a/kernel/sysfile.c b/kernel/sysfile.c
new file mode 100644
index 0000000..c250638
--- /dev/null
+++ b/kernel/sysfile.c
@@ -0,0 +1,27 @@
+.....
+uint64 sys_write(void)
+{
+    int n;
+    uint64 p;
+
+    // ignore arg 0 (fd)
+    argaddr(1, &p);
+    argint(2, &n);
+    .....
```

第一个系统调用 write

```
uint64 sys_write(void)
{
    int n;
    uint64 p;

    // ignore arg 0 (fd)
    argaddr(1, &p);
    argint(2, &n);

    return consolewrite(1, p, n);
}
```



第一个系统调用 write

```
uint64 sys_write(void)
{
    int n;
    uint64 p;

    // ignore arg 0 (fd)
    argaddr(1, &p);
    argint(2, &n);

    return consolewrite(1, p, n)
}
```

```
diff --git a/kernel/console.c b/kernel/console.c
index 7d41e3c..a15fd5e 100644
--- a/kernel/console.c
+++ b/kernel/console.c
.....
int
-consolewrite(uint64 src, int n)
+consolewrite(int user_src, uint64 src, int n)
{
    .....
    while(i < n){
        int nn = sizeof(buf);
        if(nn > n - i)
            nn = n - i;
        memmove(buf, (const void *)src+i, nn);
+        if(either_copyin(buf, user_src, src+i, nn) == -1)
+            break;
        uartwrite(buf, nn);
        i += nn;
    }
```


第一个系统调用 write

```
uint64 sys_write(void)
{
    int n;
    uint64 p;

    // ignore arg 0 (fd)
    argaddr(1, &p);
    argint(2, &n);

    return consolewrite(1, p, n)
}
```

```
diff --git a/kernel/console.c b/kernel/console.c
index 7d4...
--- a/kernel/
+++ b/kernel/
     .....
     int
-consolewrite
+consolewrite
     {
     .....
     }
     while(
         int nn = sizeof(buf);
         if(nn > n - i)
             nn = n - i;
         memmove(buf, (const void *)src+i, nn);
+         if(either_copyin(buf, user_src, src+i, nn) == -1)
+             break;
         uartwrite(buf, nn);
         i += nn;
     }
```

```
int either_copyin(void *dst, int user_src, uint64 src, uint64 len)
{
    struct proc *p = myproc();
    if(user_src){
        return copyin(p->pagetable, dst, src, len);
    } else {
        memmove(dst, (char*)src, len);
        return 0;
    }
}
```

第一个系统调用 write

```
uint64 sys_write(void)
{
    int n;
    uint64 p;

    // ignore arg 0 (fd)
    argaddr(1, &p);
    argint(2, &n);

    return consolewrite(1, p, n)
}
```

diff --git a/kernel/console.c b/kernel/console.c

index 7d4...
--- a/kernel/proc.c
+++ b/kernel/proc.c

```
.....
int
- consolewrite(uint64 p, int n)
+ consolewrite(uint64 p, int n)
{
    .....
}

while (n > 0) {
    int nn = sizeof(char);
    if (nn > n)
        nn = n;
    memmove(buf, (void *)p, nn);
    uartwrite(buf, nn);
    i += nn;
}
```

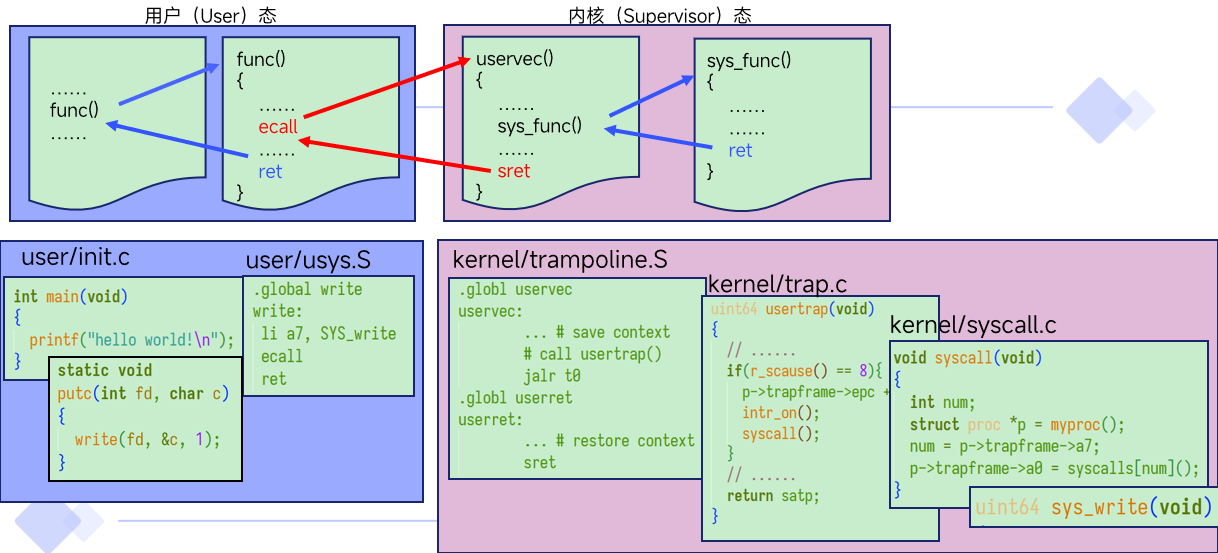
```
int either_copyin(void *dst, int user_src, uint64 src, uint64 len)
{
    struct proc *p = myproc();
    if (user_src)
        return copyin(p->pagetable, dst, src, len);
    else {
        memmove(dst, (void *)src, len);
        return 0;
    }
}

uint64 copyin(pagetable_t pagetable, char *dst, uint64 srcva, uint64 len)
{
    uint64 n, va0, pa0;

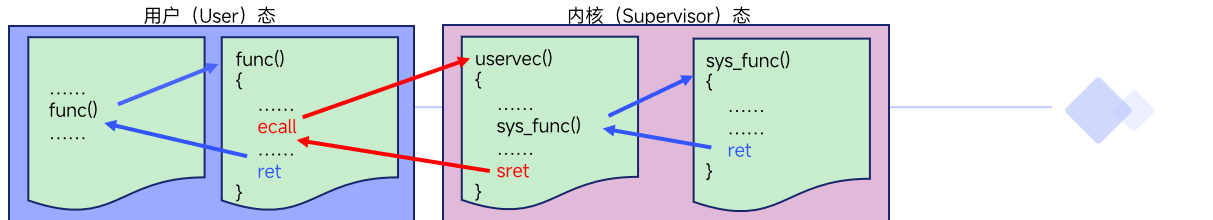
    while (len > 0) {
        va0 = PGROUNDDOWN(srcva);
        pa0 = walkaddr(pagetable, va0);
        if (pa0 == 0)
            return -1;
        n = PGSIZE - (srcva - va0);
        if (n > len)
            n = len;
        memmove(dst, (void *) (pa0 + (srcva - va0)), n);

        len -= n;
        dst += n;
        srcva = va0 + PGSIZE;
    }
    return 0;
}
```

第一个系统调用 write



第一个系统调用 write



user/init.c

```
int main(void)
{
    printf("hello world!\n");
}
```

user/usys.S

```
.global write
write:
    li a7, SYS_write
    ecall
```

kernel/trampoline.S

```
.globl uservec
uservec:
    ... # save context
    # call usertrap()
```

kernel/trap.c

```
uint64 usertrap(void)
{
    // .....
}
```

kernel/syscall.c

```
void syscall(void)
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic

xv6 kernel is booting

hart 2 starting
hart 1 starting
hello world!
```

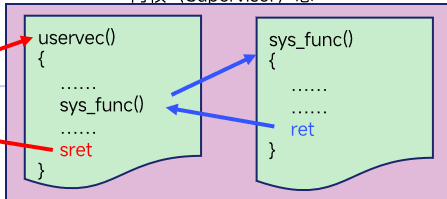
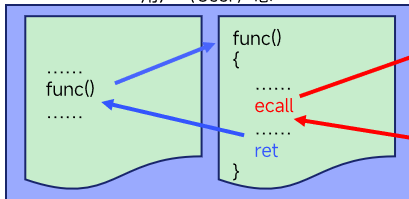
xv6 系统调用的设计与实现

- xv6 系统调用接口框架的实现
- 第一个系统调用 write
- **完善系统调用框架**
- 第二个系统调用 getpid

完善系统调用框架

用户 (User) 态

内核 (Supervisor) 态



user/initcode.c

user/usys.S

```
int main(void)
{
    printf("hello world!\n");
}

static void
putc(int fd, char c)
{
    write(fd, &c, 1);
}
```

```
.global write
write:
    li a7, SYS_write
    ecall
    ret
```

kernel/trampoline.S

```
.globl uservec
uservec:
    ... # save context
    # call usertrap()
    jalr t0

.globl userret
userret:
    ... # restore context
    sret
```

kernel/trap.c

```
uint64 usertrap(void)
{
    // .....
    if(r_scause() == 8){
        p->trapframe->epc =
            intr_on();
        syscall();
    }
    // .....
    return satp;
}
```

kernel/syscall.c

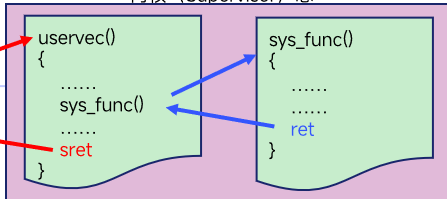
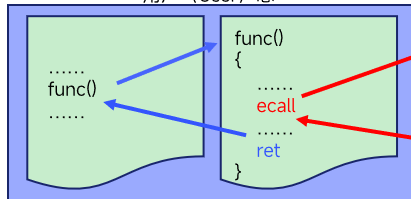
```
void syscall(void)
{
    int num;
    struct proc *p = myproc();
    num = p->trapframe->a7;
    p->trapframe->a0 = syscalls[num]();
}
```

```
uint64 sys_write(void)
```

完善系统调用框架

用户 (User) 态

内核 (Supervisor) 态



user/initcode.c

```
int main(void)
{
    printf("hello world!\n");
}
```

user/usys.S

```
.global write
write:
    li a7, SYS_write
    ecall
    ret
```

static void
putc(int fd, char c)
{
 write(fd, &c, 1);
}

重复编写

kernel/trampoline.S

```
.globl uservec
uservec:
    ... # save context
    # call usertrap()
    jalr t0

.globl userret
userret:
    ... # restore context
    sret
```

kernel/trap.c

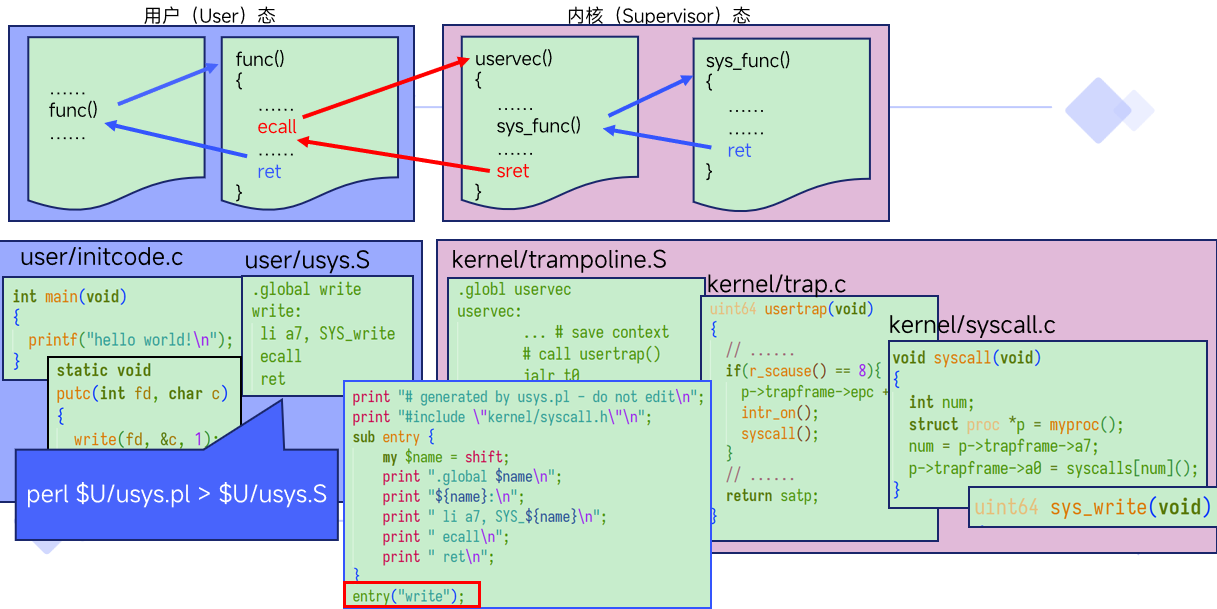
```
uint64 usertrap(void)
{
    // .....
    if(r_scause() == 8){
        p->trapframe->epc =
            intr_on();
            syscall();
    }
    // .....
    return satp;
}
```

kernel/syscall.c

```
void syscall(void)
{
    int num;
    struct proc *p = myproc();
    num = p->trapframe->a7;
    p->trapframe->a0 = syscalls[num]();
}
```

uint64 sys_write(void)

完善系统调用框架



xv6 系统调用的设计与实现

- xv6 系统调用接口框架的实现
- 第一个系统调用 write
- 完善系统调用框架
- **第二个系统调用 getpid**

第二个系统调用 getpid

getpid(2)

System Calls Manual

getpid(2)

NAME

getpid, getppid - get process identification

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

```
#include <unistd.h>
```

```
pid_t getpid(void);
```

```
pid_t getppid(void);
```

man 2 getpid

第二个系统调用 getpid

getpid(2) System Calls Manual getpid(2)

NAME
getpid, getppid - get process identification

LIBRARY
Standard C library (libc, -lc)

SYNOPSIS
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);

user/user.h
int getpid(void);

```
diff --git a/user/user.h b/user/user.h
index 05c0560..23f68c2 100644
--- a/user/user.h
+++ b/user/user.h
@@ -1,5 +1,6 @@
// system calls
int write(int, const void*, int);
+int getpid(void);
```

```
diff --git a/user/usys.pl b/user/usys.pl
index 6dc9220..e25b955 100644
--- a/user/usys.pl
+++ b/user/usys.pl
@@ -16,3 +16,4 @@ sub entry {
}

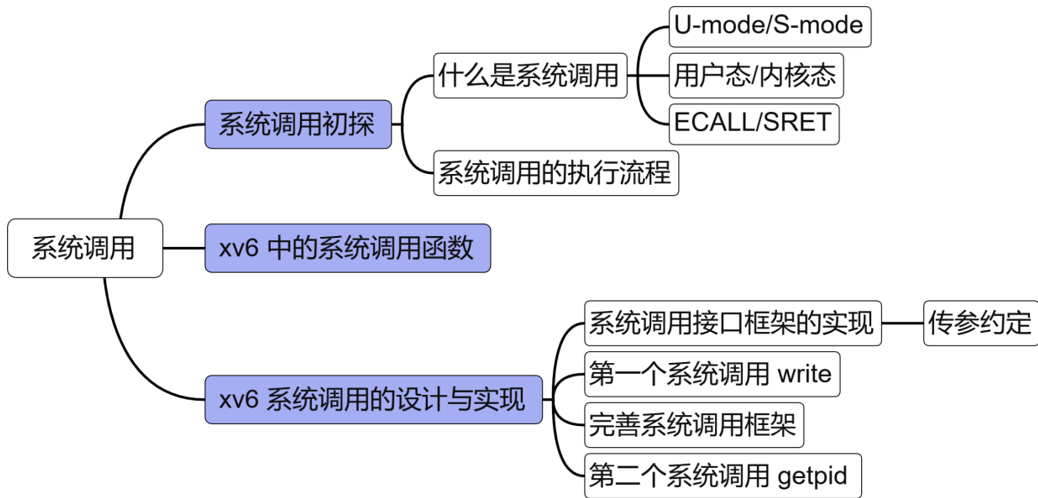
entry("write");
+entry("getpid");
```

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index 499c0f9..f0b9335 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
.....
// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
+[SYS_getpid] sys_getpid,
[SYS_write] sys_write,
};
```

```
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
new file mode 100644
index 0000000..1459405
--- /dev/null
+++ b/kernel/sysproc.c
@@ -0,0 +1,13 @@
+.....
+uint64
+sys_getpid(void)
+{
+    return myproc()->pid;
+}
```



本章总结



谢谢

